

Dziennik Projektowy:
Estymacja pozostałego czasu bezpiecznej pracy (RUL)
silnika odrzutowego z wykorzystaniem modeli
uczenia maszynowego na bazie zbioru NASA
C-MAPSS

Szymon Nowicki

Marzec – Czerwiec 2026

Spis treści

1	Wstęp i cel projektu	3
2	Charakterystyka bazy danych	3
2.1	Struktura danych	3
2.2	Zmienna docelowa – RUL	3
3	Faza 1: Eksploracyjna analiza danych (EDA)	3
3.1	Analiza statystyczna	3
3.2	Selekcja cech	4
4	Faza 2: Preprocessing	4
4.1	Normalizacja	4
4.2	Podział danych	4
4.3	Inżynieria cech – XGBoost	4
4.4	Sekwencje – LSTM	4
5	Faza 3: Modele bazowe (krok 3–6)	5
5.1	XGBoost Baseline	5
5.2	LSTM	5
5.3	CNN-LSTM	5
5.4	Wyniki na FD001	5
6	Faza 4: Ewaluacja na wszystkich zbiorach (krok 7)	5
7	Faza 5: Normalizacja per warunek operacyjny (krok 8)	6
8	Faza 6: Optymalizacja hiperparametrów – Optuna (kroki 9–10)	6
8.1	Optuna v1 (krok 9) – niepowodzenie	6
8.2	Optuna v2 (krok 9v2) – walidacja krzyżowa	6
8.3	Optuna v3 (krok 10) – augmentacja i transfer learning	7

8.4	Ewolucja najlepszych wyników	7
8.5	Finalne najlepsze wyniki	7
8.6	Ważność hiperparametrów	7
9	Faza 7: Analiza i walidacja wyników	8
9.1	Accuracy Window	8
9.2	Test Wilcoxona	8
9.3	Analiza rezyduów	8
9.4	Analiza kosztowa	8
9.5	Interpretowalność	8
9.6	Dodatkowe wizualizacje	8
10	Faza 8: Testy odporności (robustness)	9
10.1	Szum sensorów	9
10.2	Dropout sensorów	9
10.3	Skrócona historia LSTM	9
10.4	Bootstrap Confidence Intervals	9
10.5	Cross-dataset generalizacja	10
11	Wnioski	10
12	Struktura kodu	11
13	Narzędzia i środowisko	11

1 Wstęp i cel projektu

Niniejszy dziennik stanowi systematyczną dokumentację prac nad projektem realizowanym w ramach kursu *Zastosowanie modeli sztucznej inteligencji w automatyce*. Dokument zawiera opis podejmowanych decyzji projektowych, strukturę modeli, przebieg optymalizacji oraz analizę uzyskanych wyników.

Celem projektu jest opracowanie systemu klasy **Predictive Maintenance** opartego na uczeniu maszynowym. Zadanie polega na estymacji pozostałego czasu bezpiecznej pracy (RUL – *Remaining Useful Life*) silników turbinowych na podstawie danych z czujników procesowych. Porównano dwa podejścia: klasyczne uczenie maszynowe (XGBoost) oraz sieci rekurencyjne (LSTM).

2 Charakterystyka bazy danych

W projekcie wykorzystano zbiór danych NASA C-MAPSS (*Commercial Modular Aero-Propulsion System Simulation – Turbofan Engine Degradation Simulation Dataset*). Zbiór zawiera symulowane dane z czujników silników odrzutowych pracujących aż do wystąpienia awarii.

2.1 Struktura danych

Zbiór składa się z 4 podzbiorów o rosnącej złożoności:

Tabela 1: Charakterystyka podzbiorów C-MAPSS

Zbiór	Warunki oper.	Tryby awarii	Silniki (train)	Silniki (test)
FD001	1	1	100	100
FD002	6	1	260	259
FD003	1	2	100	100
FD004	6	2	248	248

Każdy wiersz zawiera: identyfikator silnika, numer cyklu, 3 ustawienia operacyjne oraz odczyty z 21 czujników (m.in. temperatura, ciśnienie, prędkość obrotowa, przepływ paliwa).

2.2 Zmienna docelowa – RUL

Remaining Useful Life obliczany jako $RUL = \max_cycle - current_cycle$, z ograniczeniem górnym (*piece-wise linear*) $RUL_{clip} = 125$ cykli. Uzasadnienie: w początkowej fazie życia silnika degradacja jest niewidoczna w danych sensorycznych, więc rozróżnianie między RUL=200 a RUL=300 jest bezcelowe.

3 Faza 1: Eksploracyjna analiza danych (EDA)

3.1 Analiza statystyczna

Analiza zbioru FD001 (100 silników, 20 631 wierszy) wykazała:

- Brak wartości brakujących we wszystkich podzbiorach.
- Średni czas życia silnika: 206 cykli (min: 128, max: 362).
- 7 sensorów z zerową wariancją (sensor_1, 5, 6, 10, 16, 18, 19) – usunięte.
- 3 sensory z bardzo niską zmiennością (sensor_2, 8, 13) – usunięte na FD001.
- Sensor_14 redundantny (korelacja 0.96 z sensor_9) – usunięty na FD001.

3.2 Selekcja cech

Zachowano 14–18 sensorów (w zależności od podzbioru – automatyczna selekcja na bazie wariancji). Najsilniejsze korelacje z RUL na FD001: sensor_11 ($r = -0.70$), sensor_4 ($r = -0.68$), sensor_12 ($r = +0.67$).

4 Faza 2: Preprocessing

4.1 Normalizacja

Wersja v1 (kroki 1–7): Globalny MinMaxScaler(0,1) dopasowany wyłącznie na zbiorze treningowym.

Wersja v2 (kroki 8–10): Normalizacja per warunek operacyjny – kluczowa poprawka dla FD002/FD004. Algorytm KMeans ($k = 6$) grupuje dane po ustawieniach operacyjnych, następnie osobny MinMaxScaler per klaster. Uzasadnienie: w FD002/FD004 silnik pracuje na 6 różnych reżimach lotu (różna wysokość, prędkość Mach, throttle). Sensory skaczą przy zmianie reżimu – nie z powodu degradacji, lecz z powodu fizyki. Globalny scaler nie rozróżnia tych skoków od prawdziwej degradacji.

4.2 Podział danych

Podział train/validation po `unit_id` (80/20), nie po wierszach – zapobiega wyciekowi danych (data leakage) z tego samego silnika do obu zbiorów. W wersji z Optuna zastosowano 3-krotną walidację krzyżową (K-Fold CV) również po `unit_id`.

4.3 Inżynieria cech – XGBoost

Dla XGBoost wygenerowano cechy statystyczne w oknach czasowych $w \in \{5, 10, 20\}$:

- **Podstawowe:** rolling mean, std, min, max, trend (diff).
- **Rozszerzone (v2+):** EMA (exponential moving average), percentyle 10/90, coefficient of variation, ratio między sensorami.

Łącznie 150–400 cech w zależności od podzbioru i wersji.

4.4 Sekwencje – LSTM

Sekwencje o długości L timestepów (domyślnie 30, po Optuna: 40) z przesuwным oknem. Dla silników z historią krótszą niż L – zero-padding od początku. Test: ostatnia sekwencja L cykli per silnik.

5 Faza 3: Modele bazowe (krok 3–6)

5.1 XGBoost Baseline

Gradient Boosted Trees z ręcznie dobranymi hiperparametrami: `n_estimators=500`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`, `early_stopping_rounds=30`.

5.2 LSTM

Dwie warstwy LSTM (64 → 32 jednostki), dropout 0.3, warstwa Dense(32) → Dense(1). Optimizer Adam ($lr = 0.001$), loss MSE, scheduler ReduceLROnPlateau. Gradient clipping ($\max_norm = 1.0$).

5.3 CNN-LSTM

Architektura hybrydowa: Conv1D(32) → Conv1D(64) → MaxPool1D(2) → LSTM(64) → LSTM(32) → Dense. Konwolucja miała wyłapywać lokalne wzorce (nagłe skoki sensorów). Wynik: porównywalny lub gorszy niż czysty LSTM na wszystkich zbiorach – degradacja w C-MAPSS jest gładka i monotonna, konwolucja nie wnosi wartości dodanej. **Porzucony w kolejnych iteracjach.**

5.4 Wyniki na FD001

Tabela 2: Porównanie modeli bazowych – FD001

Model	RMSE	MAE	R ²	NASA Score
XGBoost	16.74	12.07	0.83	586
LSTM	14.13	10.82	0.88	350
CNN-LSTM	14.57	10.96	0.87	371

NASA Scoring Function (asymetryczna kara): $d = \hat{y} - y$; jeśli $d < 0$: $s = e^{-d/13} - 1$ (za wcześnie – mniejsza kara); jeśli $d \geq 0$: $s = e^{d/10} - 1$ (za późno – większa kara, bo awaria jest niebezpieczna).

6 Faza 4: Ewaluacja na wszystkich zbiorach (krok 7)

Uruchomienie pełnego pipeline’u na FD001–FD004 ujawniło kluczowy problem: LSTM dominuje na zbiorach z 1 warunkiem operacyjnym (FD001, FD003), ale **załamuje się na FD002/FD004** (6 warunków). NASA Score LSTM na FD004: 9 691 – katastrofalny wynik spowodowany szumem z przełączania reżimów lotu.

Tabela 3: Wyniki v1 na wszystkich zbiorach

	FD001	FD002	FD003	FD004
XGBoost RMSE	16.7	18.6	16.4	19.8
LSTM RMSE	13.3	18.8	12.6	20.3
LSTM NASA	298	4 276	288	9 691

7 Faza 5: Normalizacja per warunek operacyjny (krok 8)

Wprowadzenie normalizacji per klaster operacyjny (KMeans $k = 6$ na `op_settings`) dało największą pojedynczą poprawę w całym projekcie:

Tabela 4: Wpływ normalizacji per warunek na NASA Score LSTM

Zbiór	Przed (v1)	Po (v2)
FD002	4 276	1 248 (-71%)
FD004	9 691	4 004 (-59%)

Wniosek: Preprocessing okazał się ważniejszy niż architektura modelu. Zmiana LSTM $\rightarrow$ BiLSTM+Attention dała marginalną poprawę, ale normalizacja per warunek zmieniła wszystko.

8 Faza 6: Optymalizacja hiperparametrów – Optuna (kroki 9–10)

8.1 Optuna v1 (krok 9) – niepowodzenie

Pierwsza próba automatycznego strojenia hiperparametrów:

- 20 prób Optuna, jeden split train/val (80/20).
- Szeroka przestrzeń: `hidden` $\in \{32, 64, 128, 256\}$, `layers` $\in \{1, 2, 3\}$, `seq_length` $\in \{20, 30, 40, 50\}$.
- 60 epok w fazie szukania.
- **Rezultat: LSTM FD004 NASA = 27 905** – katastrofa.

Przyczyny: za duża przestrzeń na za mało prób, niedotrenowane modele w fazie szukania, jeden split daje niestabilną ocenę.

8.2 Optuna v2 (krok 9v2) – walidacja krzyżowa

Kluczowe poprawki:

- **3-krotna walidacja krzyżowa (K-Fold CV)** po `unit_id` – każda próba Optuna trenuje 3 razy na różnych podziałach i uśrednia wynik. Stabilniejsza ocena.
- **MedianPruner** – co 10 epok sprawdza czy próba jest gorsza od mediany dotychczasowych i ucina ją wcześniej. Oszczędza czas na złe konfiguracje.
- Zawężona przestrzeń: `hidden` $\in \{64, 128\}$, `layers` $\in \{1, 2\}$, `seq_length` $\in \{30, 40\}$.
- 80 epok w fazie szukania, 30 prób XGBoost, 25 prób LSTM.
- Ensemble 5 modeli z różnymi seedami.
- **Rezultat: FD004 NASA = 4 970** (vs 27 905 w v1).

8.3 Optuna v3 (krok 10) – augmentacja i transfer learning

Dalsze ulepszenia:

- 50 prób XGBoost, 40 prób LSTM.
- **Data augmentation:** szum Gaussowski ($\sigma = 0.02$) + time jitter ($p = 0.1$) – $3\times$ więcej danych treningowych. Augmentacja tylko w fazie ensemble (nie w fazie szukania – za wolno).
- **OneCycleLR scheduler** zamiast CosineAnnealing – agresywnie rośnie LR w pierwszych 30% epok, potem powoli obniża.
- **Transfer learning:** pretrain na zbiorze z tymi samymi warunkami operacyjnymi (FD002 $\rightarrow$ FD004). Ograniczony efekt – różna liczba sensorów między FD001 (14) a FD003 (15) uniemożliwia transfer wag.
- Ensemble 10 modeli z różnymi seedami.

8.4 Ewolucja najlepszych wyników

Tabela 5: Najlepsze wyniki LSTM (RMSE / NASA Score) w kolejnych iteracjach

	FD001	FD002	FD003	FD004
v1 (baseline)	13.3 / 298	18.8 / 4 276	12.6 / 288	20.3 / 9 691
Optuna v1	13.4 / 322	21.7 / 11 879	12.0 / 285	28.1 / 27 905
Optuna v2 (K-Fold)	12.8 / 284	16.2 / 1 725	12.2 / 261	19.1 / 4 970
Optuna v3 (final)	12.1 / 266	15.1 / 1 266	12.1 / 316	19.4 / 3 782

8.5 Finalne najlepsze wyniki

Tabela 6: Najlepsze uzyskane wyniki (RMSE / NASA Score) – pogrubiony = zwycięzca per zbiór

	FD001	FD002	FD003	FD004
XGBoost	16.6 / 576	15.8 / 1 264	17.1 / 1 191	18.2 / 2 161
LSTM	12.1 / 266	16.4 / 2 233	12.1 / 316	19.4 / 3 782

LSTM dominuje na zbiorach z 1 warunkiem operacyjnym (FD001, FD003). XGBoost wygrywa na zbiorach z 6 warunkami (FD002, FD004). Nie ma jednego najlepszego modelu.

8.6 Ważność hiperparametrów

Analiza korelacji hiperparametrów z wynikiem:

- **LSTM:** `seq_length` najważniejszy na FD001/FD002/FD004 (korelacja 0.96–1.00). Na FD003 dominuje `dense` (0.68) i `n_layers` (0.55). `batch_size` prawie nieistotny (< 0.05).

- **XGBoost:** `max_depth` i `learning_rate` stabilnie na szczycie (0.5–0.9). Na FD004 (najtrudniejszy) większość parametrów istotna (0.3–0.6).

9 Faza 7: Analiza i walidacja wyników

9.1 Accuracy Window

Procent predykcji mieszczących się w oknie $\pm N$ cykli od prawdy:

Tabela 7: Accuracy Window – LSTM

	± 5	± 10	± 15	± 20
FD001	50%	70%	80%	89%
FD002	37%	63%	76%	83%
FD003	48%	70%	84%	90%
FD004	38%	57%	71%	76%

9.2 Test Wilcoxona

Sparowany test Wilcoxona (signed-rank) na błędach bezwzględnych per silnik testowy. Wynik: różnica między XGBoost a LSTM jest **statystycznie istotna** ($p < 0.01$) na **wszystkich 4 zbiorach**. LSTM ma niższą medianę $|\text{błędu}|$ wszędzie.

9.3 Analiza rezyduów

Negatywna korelacja błędu z rzeczywistym RUL na wszystkich zbiorach ($r \in [-0.19, -0.41]$): oba modele mają tendencję do zaniżania RUL przy wysokich wartościach – predykcja „za wcześnie” dla zdrowych silników. Jest to efekt piece-wise linear clippingu na $\text{RUL}_{\text{clip}} = 125$.

9.4 Analiza kosztowa

Scenariusz: wczesna wymiana $\times 1$, awaria $\times 10$. LSTM generuje niższy łączny koszt eksploatacyjny na wszystkich zbiorach (np. FD001: LSTM 5 243 vs XGBoost 7 891).

9.5 Interpretowalność

- **SHAP (XGBoost):** Najważniejsze cechy to rolling mean `sensor_4` i `sensor_15` w oknie 10 cykli. Wysoka wartość `sensor_4_mean_10` obniża predykcję RUL (model rozpoznaje zaawansowaną degradację).
- **Gradient Saliency (LSTM):** Model zwraca największą uwagę na ostatnie 5–10 timestepów w sekwencji oraz na sensory 4, 11, 12 – zgodne z wynikami EDA i SHAP.

9.6 Dodatkowe wizualizacje

Wygenerowano również: CDF błędów (kumulatywny rozkład), błąd per zakres RUL (box ploty), Bland-Altman (zgodność modeli z Limits of Agreement), mapa zgodności per silnik

(gdzie oba modele trafiają, gdzie się mylą), ROC/AUC dla binarnego alertu awarii (≤ 30 cykli), radar chart profilu modeli.

10 Faza 8: Testy odporności (robustness)

10.1 Szum sensorów

Dodanie szumu Gaussowskiego ($\sigma = 0 \rightarrow 0.3$) do danych testowych. Wynik: **LSTM jest odporniejszy niż XGBoost** na wszystkich zbiorach. Przy $\sigma = 0.1$ LSTM osiąga RMSE ~ 18 –25, XGBoost ~ 24 –28. Przy $\sigma = 0.3$ oba modele degradują do RMSE ~ 29 –37, ale LSTM zawsze niżej. Spodziewano się odwrotnego wyniku (rolling statistics XGBoosta powinny wygładzać szum), ale LSTM nauczył się wewnętrznej reprezentacji odporniejszej na perturbacje.

10.2 Dropout sensorów

Zerowanie jednego sensora i pomiar wzrostu RMSE (Δ RMSE):

- **Krytyczne** (Δ RMSE > 5): sensor_11, sensor_12, sensor_4 – na wszystkich zbiorach. Usunięcie sensor_11 powoduje wzrost RMSE o 6–14 punktów ($\sim 50\%$ degradacji).
- **Umiarkowane** (Δ RMSE 1–5): sensor_2, sensor_7, sensor_15.
- **Redundantne** (Δ RMSE < 0.5): sensor_3, sensor_14, sensor_20.

Wyniki zgodne z analizą SHAP i korelacjami z EDA – te same sensory wychodzą jako najważniejsze.

10.3 Skrócona historia LSTM

Test LSTM z obciętą historią ($5 \rightarrow 40$ timestepów). Wynik: RMSE **rośnie** od 5 do 15–25 timestepów, potem spada do minimum przy 40. Wyjaśnienie: model trenowany na 40 timestepach. Przy 15 timestepach + 25 zer paddingu, zera „mylą” model – widzi nagły skok z zera do wartości. Przy 5 timestepach prawie same zera, więc model predykuje średnią (niższy RMSE, ale bezużyteczna predykcja). **Model naprawdę potrzebuje pełnej historii 35–40 cykli.**

10.4 Bootstrap Confidence Intervals

1000 resamplings zbioru testowego $\rightarrow$ 95% przedziały ufności dla RMSE:

Tabela 8: Bootstrap 95% CI

Zbiór	XGBoost	LSTM	CI nakładają się?
FD001	16.60 [13.97, 19.29]	12.10 [9.88, 14.38]	NIE – LSTM pewnie lepszy
FD002	15.80 [14.19, 17.32]	16.36 [13.77, 19.16]	TAK – brak pewnego zwycięzcy
FD003	17.04 [13.65, 20.26]	11.95 [9.51, 14.44]	NIE – LSTM pewnie lepszy
FD004	18.12 [16.12, 20.01]	19.35 [16.21, 22.45]	TAK – brak pewnego zwycięzcy

Na FD001 i FD003 przedziały CI się nie nakładają – LSTM jest pewnie lepszy. Na FD002 i FD004 CI się nakładają – nie można jednoznacznie wskazać zwycięzcy (choć XGBoost ma niższą średnią).

10.5 Cross-dataset generalizacja

Trening na jednym zbiorze, test na innym. Wyniki degradacji RMSE:

Tabela 9: Cross-dataset generalizacja LSTM

Para	Natywny RMSE	Cross RMSE	Degradacja
FD001 → FD003	12.1	51.9	+329%
FD003 → FD001	12.1	56.2	+364%
FD002 → FD004	19.4	32.5	+68%
FD004 → FD002	16.4	40.5	+147%

Generalizacja jest słaba – szczególnie między zbiorami z różnymi trybami awarii (FD001 ↔ FD003: > 300%). Transfer między zbiorami z tymi samymi warunkami operacyjnymi (FD002 ↔ FD004) jest lepszy, ale wciąż +68–147%. **Wniosek:** modele są specjalistami, nie generalistami – każdy zbiór wymaga dedykowanego treningu.

11 Wnioski

1. **Nie ma jednego najlepszego modelu.** LSTM dominuje na prostych zbiorach (1 warunek operacyjny, FD001/FD003), XGBoost na złożonych (6 warunków, FD002/FD004). Potwierdzone testem Wilcozona ($p < 0.01$) i bootstrap CI.
2. **Preprocessing ważniejszy niż architektura.** Normalizacja per warunek operacyjny (KMeans) dała ~60% poprawę NASA Score na FD002/FD004. Zmiana architektury (LSTM → BiLSTM+Attention, CNN-LSTM) dała < 5%.
3. **Walidacja krzyżowa jest kluczowa** dla Optuna – jeden split train/val prowadzi do niestabilnych i potencjalnie katastrofalnych wyników (NASA 27 905 vs 4 970 po K-Fold CV).
4. **Sequence length to najważniejszy hiperparametr LSTM** (korelacja z wynikiem > 0.95 na 3/4 zbiorów). Zmiana 30 → 40 timestepów dała więcej niż strojenie architektury. Test skróconej historii potwierdził: model potrzebuje pełnych 35–40 cykli.
5. **LSTM jest odporniejszy na szum sensorów** niż XGBoost – wbrew oczekiwaniom. Ensemble 10 modeli z augmentacją danych dodatkowo stabilizuje predykcje.
6. **Sensor_11, sensor_12, sensor_4 są krytyczne** – ich usunięcie degradowało RMSE o 50%. Zgodne z SHAP, gradient saliency i EDA.
7. **Modele nie generalizują między zbiorami** – cross-dataset RMSE rośnie o 68–364%. Każdy scenariusz operacyjny wymaga dedykowanego modelu.

8. **Data augmentation pomaga umiarkowanie** ($\sim 5\%$ poprawa RMSE), **transfer learning ma ograniczone zastosowanie** – różna liczba sensorów między zbiorami uniemożliwia bezpośredni transfer wag.

12 Struktura kodu

Finalny kod składa się z modularnych skryptów uruchamianych z jednego pliku `run.py`:

Tabela 10: Struktura plików projektu

Plik	Opis	Czas
<code>run.py</code>	Menu główne – uruchamia wybrane fazy	–
<code>shared.py</code>	Wspólne funkcje, modele, konfiguracja	–
<code>10a_train.py</code>	Trening XGBoost + LSTM (Optuna, K-Fold CV)	~ 50 min
<code>10b_plots.py</code>	Wykresy porównawcze (53–60)	~ 30 sek
<code>10c_analysis.py</code>	Analiza i walidacja (61–73)	~ 5 min
<code>10d_robustness.py</code>	Testy odporności (74–78)	~ 10 min

Łącznie wygenerowano **78 wykresów** w 4 kategoriach:

- Wykresy 01–11: EDA i preprocessing.
- Wykresy 12–35: Modele bazowe (XGBoost, LSTM, CNN-LSTM) i porównania.
- Wykresy 36–60: Optymalizacja v1/v2/v3 (heatmaps, scatter, parallel coordinates, ważność hiperparametrów, optimization history).
- Wykresy 61–78: Analiza (accuracy window, rezyduały, Wilcoxon, koszt, SHAP, saliency, degradacja, CDF, box ploty, Bland-Altman, mapa zgodności, ROC/AUC, radar) i odporność (szum, dropout, historia, bootstrap CI, cross-dataset).

13 Narzędzia i środowisko

- **Język:** Python 3.10
- **ML:** XGBoost, PyTorch (LSTM), scikit-learn
- **Optymalizacja:** Optuna (TPESampler, MedianPruner, K-Fold CV)
- **Interpretowalność:** SHAP (TreeExplainer), gradient saliency
- **Statystyka:** SciPy (Wilcoxon signed-rank test, bootstrap CI, Pearson correlation)
- **GPU:** NVIDIA GeForce RTX 4060 Ti (16 GB VRAM), CUDA 13.0, PyTorch cu126
- **Dane:** NASA C-MAPSS via kagglehub (Kaggle)